Task: Website hosting through S3 bucket and configuration of backend.

Task Details:

- Host the **frontend (HTML/CSS/JS)** on **Amazon S3** using **Static Website Hosting**.
- Deploy the **backend application/API** on a **private EC2 instance** behind an **Application Load Balancer (ALB)**.
- Ensure the backend EC2 instance has **no public access** and is reachable **only via the ALB**.
- Configure **Amazon CloudFront** in front of the S3 bucket to serve the frontend.
- Update the frontend code to call the backend using the **ALB DNS name** (Example: http://<ALB-DNS>/api).

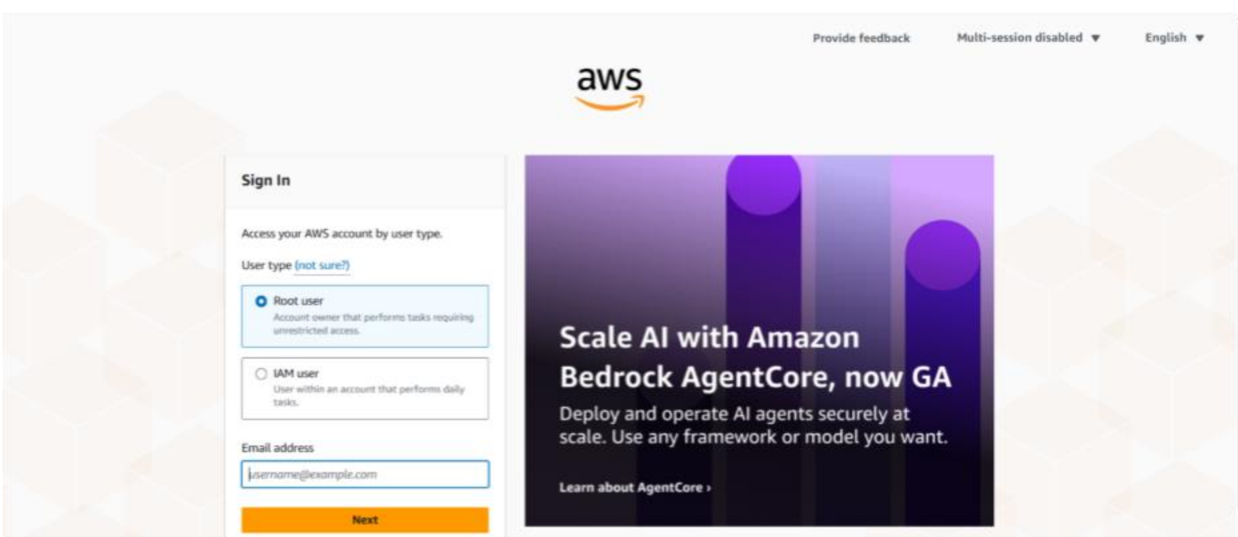
To cross verify your setup ,you can used this command

curl http://<ALB-DNS>

Solution:

Login to your AWS account with credentials and required verification.

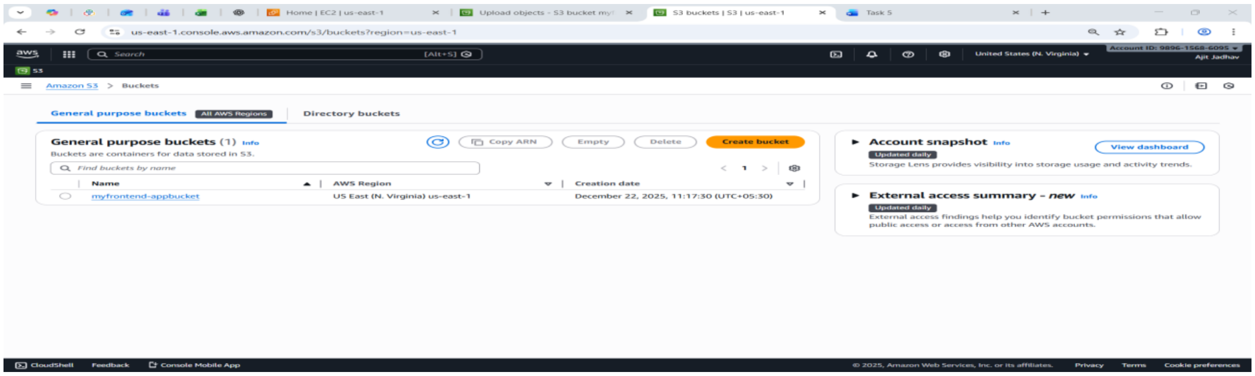
1. (https://signin.aws.amazon.com/)
   My account – AWS Management Console- Sign in



2. Create S3 Bucket

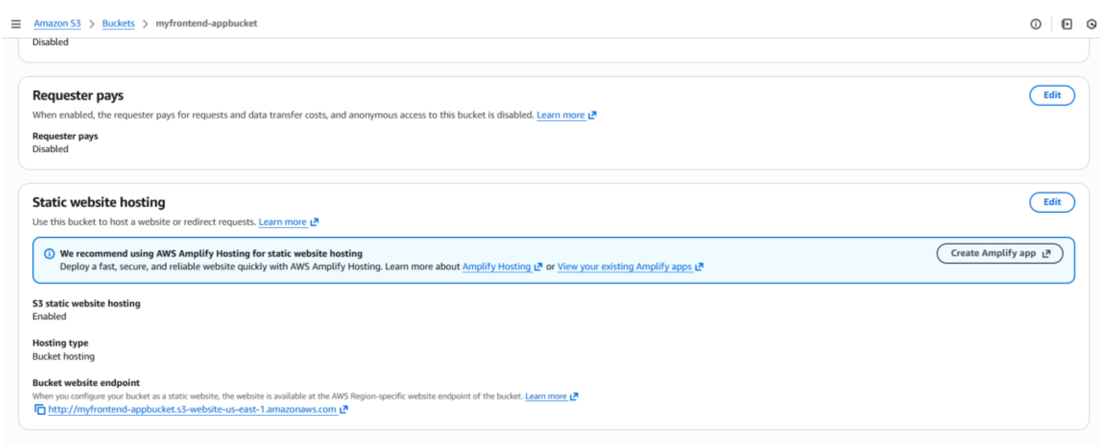
- Goto S3- create bucket-

- Enter:
- Bucket name: *myfrontedend-appbucket*
- Region: same as your ALB
- **Uncheck** “Block all public access”
- Create the bucket



3. Enable Static website hosting

- Open your bucket
- Go to **Properties**
- Scroll to **Static website hosting**
- Enable it
- Set:
- Index document: *index.html*
- Save changes



4. Upload Files

1. Go to Objects → Upload

2. Upload: Index.html

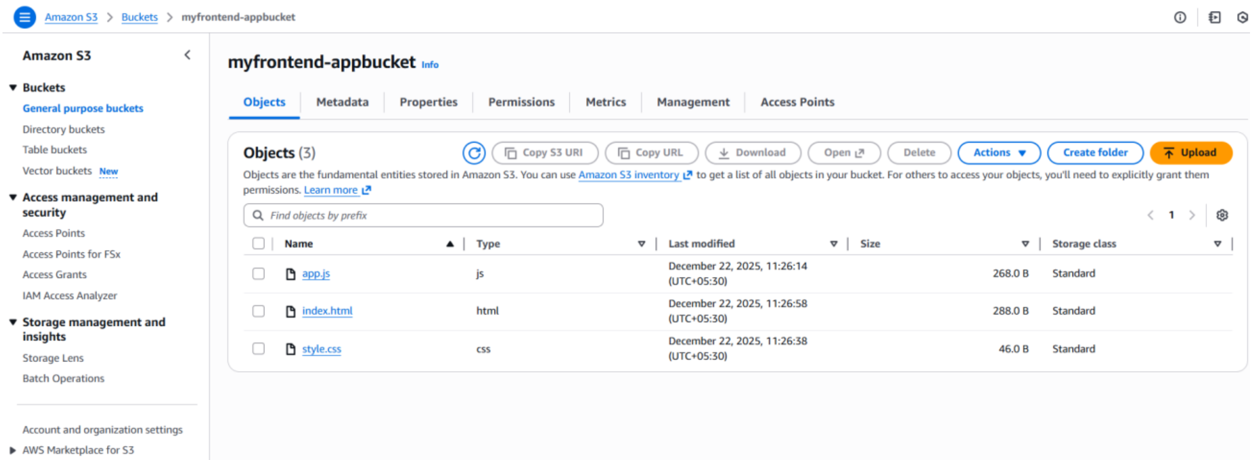
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Static Frontend - Cloud Project</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="container">
        <h1>Static Frontend (S3 + CloudFront)</h1>
        <p>This frontend is hosted on Amazon S3 and served via CloudFront.</p>

        <button onclick="callBackend()">Call Backend API</button>

        <div id="response-box">
            <h3>Backend Response:</h3>
            <p id="api-response">Waiting for response...</p>
        </div>
    </div>

    <script src="app.js"></script>
</body>
</html>
```

App.j

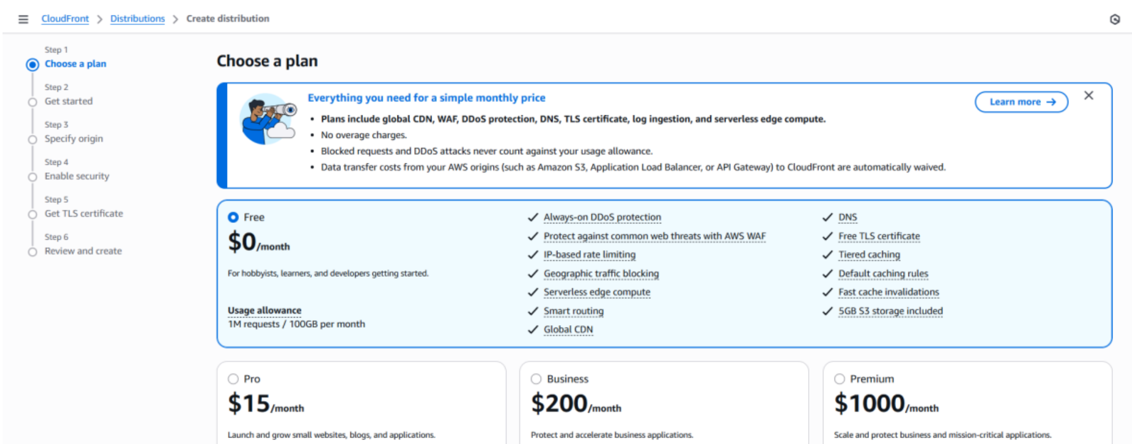
```
function callBackend() {
    // Replace <ALB-DNS> with your actual ALB DNS name
    const backendUrl = "http://taskappjs-1714280081.us-east-1.elb.amazonaws.com/api";

    fetch(backendUrl)
        .then(response => {
            if (!response.ok) {
                throw new Error("Backend not reachable");
            }
            return response.text();
        })
        .then(data => {
            document.getElementById("api-response").innerText = data;
        })
        .catch(error => {
            document.getElementById("api-response").innerText =
                "Error connecting to backend";
            console.error(error);
        });
}
```

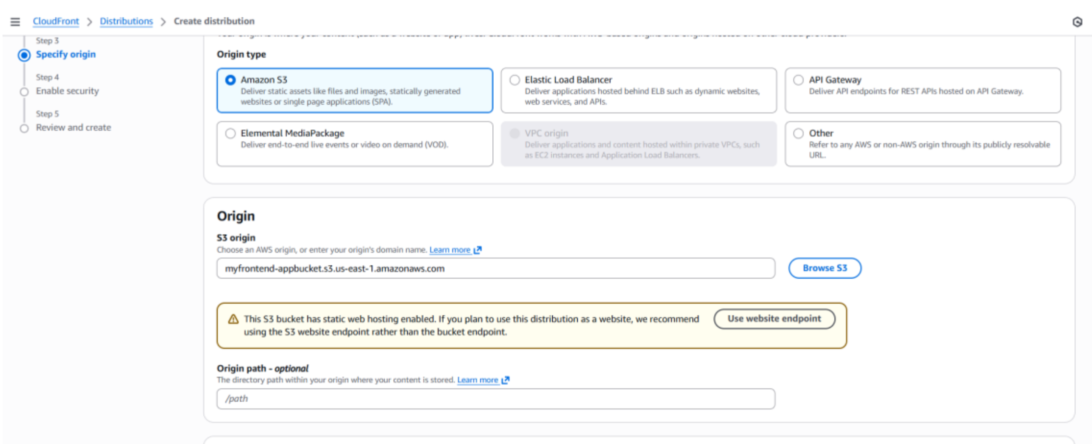
Style.css

```css
body {
    font-family: Arial, sans-serif;
    background-color: #f4f6f8;
    margin: 0;
    padding: 0;
}

.container {
    width: 60%;
    margin: 80px auto;
    padding: 30px;
    background-color: #ffffff;
    text-align: center;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

button {
    background-color: #0073e6;
    color: white;
    border: none;
    padding: 12px 25px;
    font-size: 16px;
    cursor: pointer;
    border-radius: 5px;
}

button:hover {
    background-color: #005bb5;
}

#response-box {
    margin-top: 30px;
    padding: 15px;
    border: 1px solid #ddd;
    border-radius: 5px;
}

#api-response {
    color: green;
    font-weight: bold;
}
```

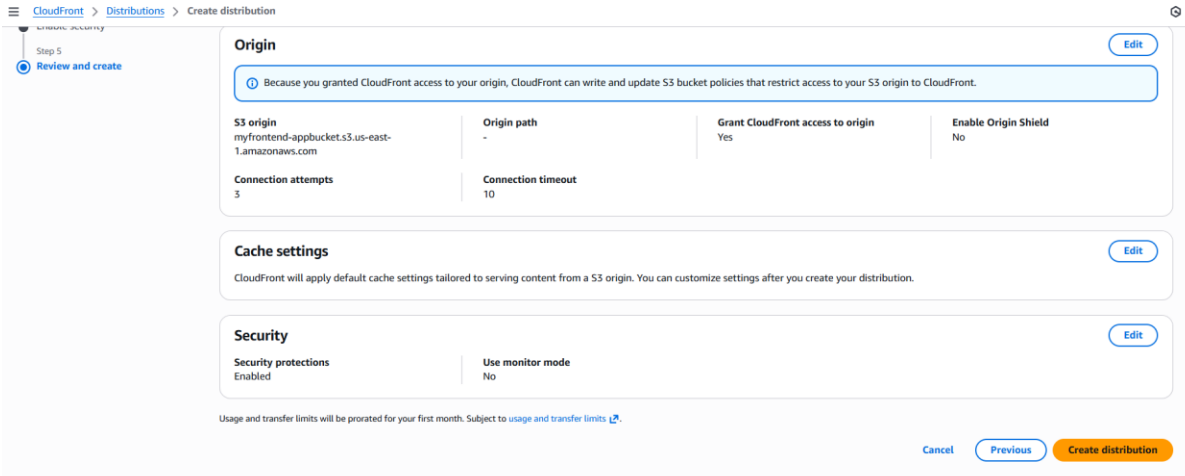


5. Create CloudFront Distribution





And create distribution



4. Deploy Backend on a Private EC2 Behind ALB

- Go to EC2 console - Launch instance.
- Select an OS (e.g., Amazon Linux 2).
- VPC & Subnet:
- Choose a **private subnet** (no internet gateway).
- Security group: allow inbound traffic only from the ALB (we will configure this later).
- Key pair: select/create one for SSH access if needed.
- Deploy Backend Application
- SSH into your EC2 instance (if allowed through VPN or bastion host).
- Install your backend runtime (Node.js, Python, etc.).
- Start your backend on a port

Deploy your app.js

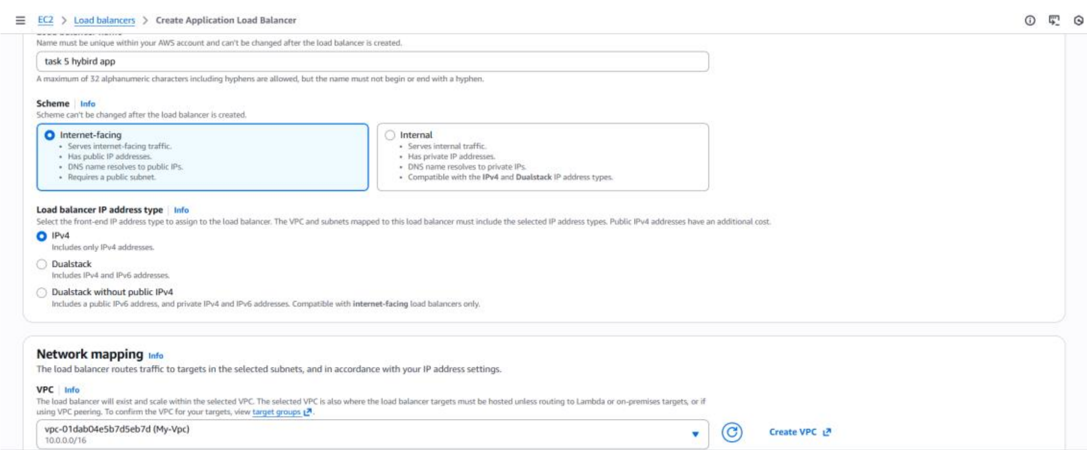
```
  GNU nano 8.3
// app.js
const express = require("express");
const app = express();

app.get("/api", (req, res) => {
  res.json({ message: "Backend is working" });
});

app.listen(3000, '0.0.0.0', () => {
  console.log("Server running on port 3000");
});
```

Run to check working or not.

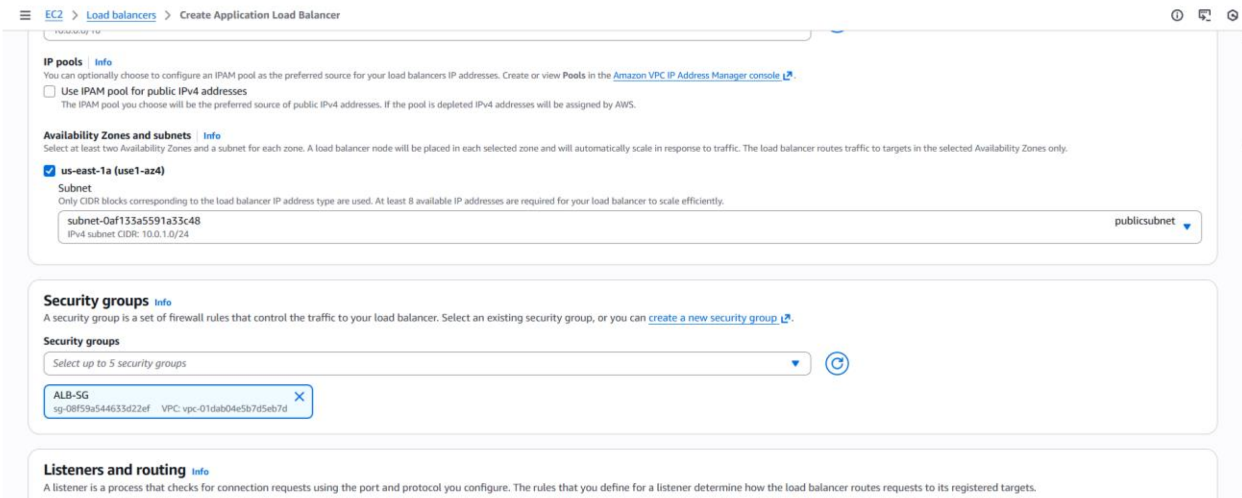
```
{"message":"Backend is working"}[ec2-user@ip-10-0-2-96 ~]$ node app.js
Server running on port 3000
[ec2-user@ip-10-0-2-96 ~]$
```

5. Create an Application Load Balancer (ALB)

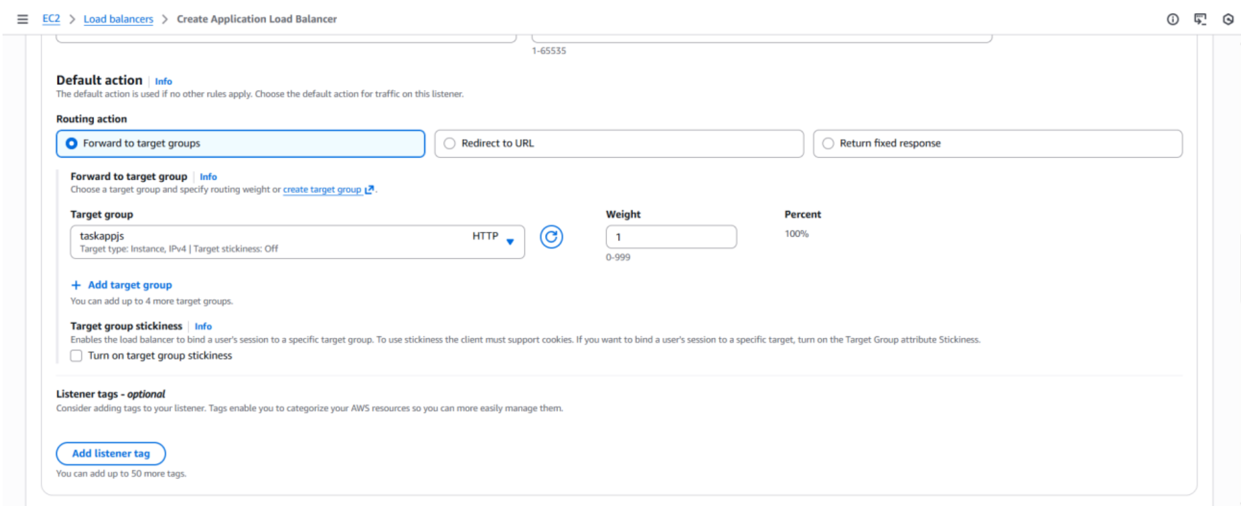
- Go to EC2 → Load Balancers → Create Load Balancer → Application Load Balancer.
- Scheme: internet-facing if the frontend needs to access it publicly.
- Listeners: HTTP 80 (or HTTPS 443 if you have a certificate).
- Subnets: choose public subnets.
- Security group: allow inbound HTTP/HTTPS from anywhere or restrict to frontend CloudFront if needed.



Select subnets and security group for ALB

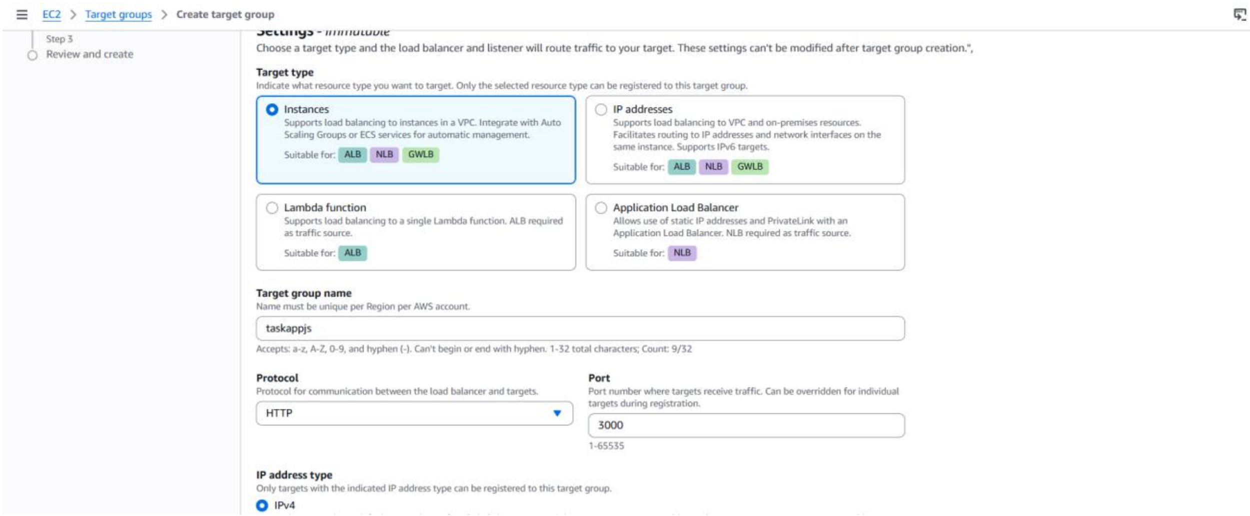


Select forward to target groups and crate a new target group

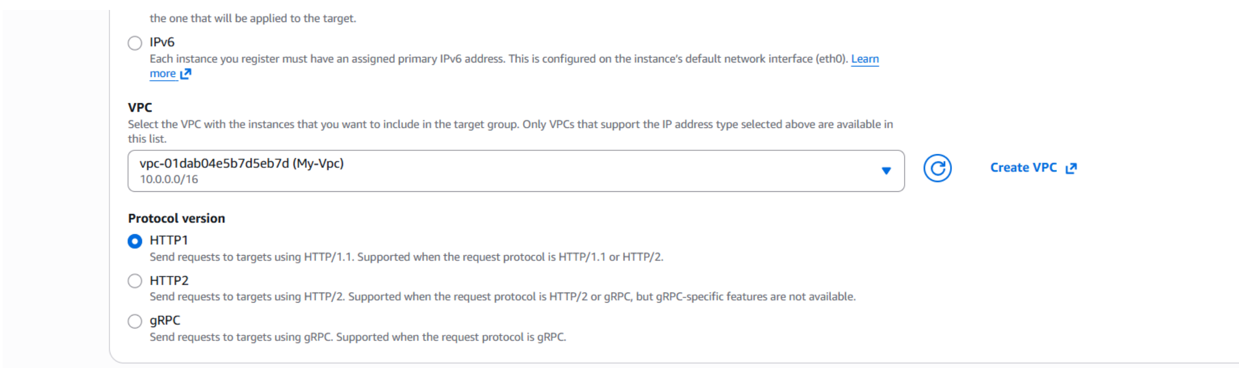


6. Target Group for EC2
    - Create a target group → Type: instance → Protocol: HTTP → Port: 3000 (your backend port).
    - Register your EC2 instance with this target group.
    - Attach the target group to the ALB listener

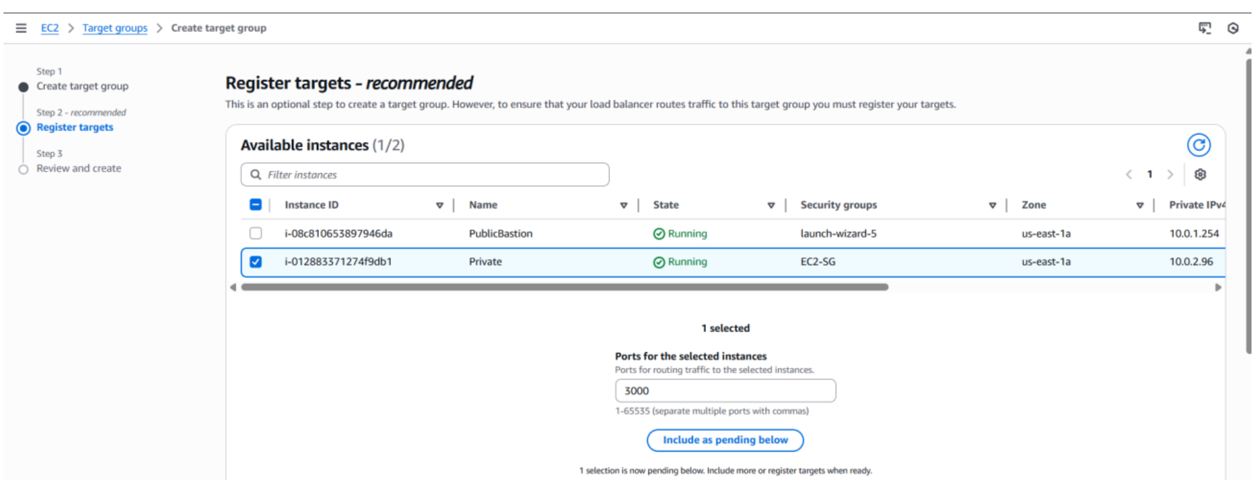
Select target type - instances



Select your VPC



Register target – private instance



Attach this target group to ALB and register our private instance to target.

7. Verify backend access
    - test backend directly from a machine with internet access (frontend should call ALB)

      curl http://<ALB-DNS>/api

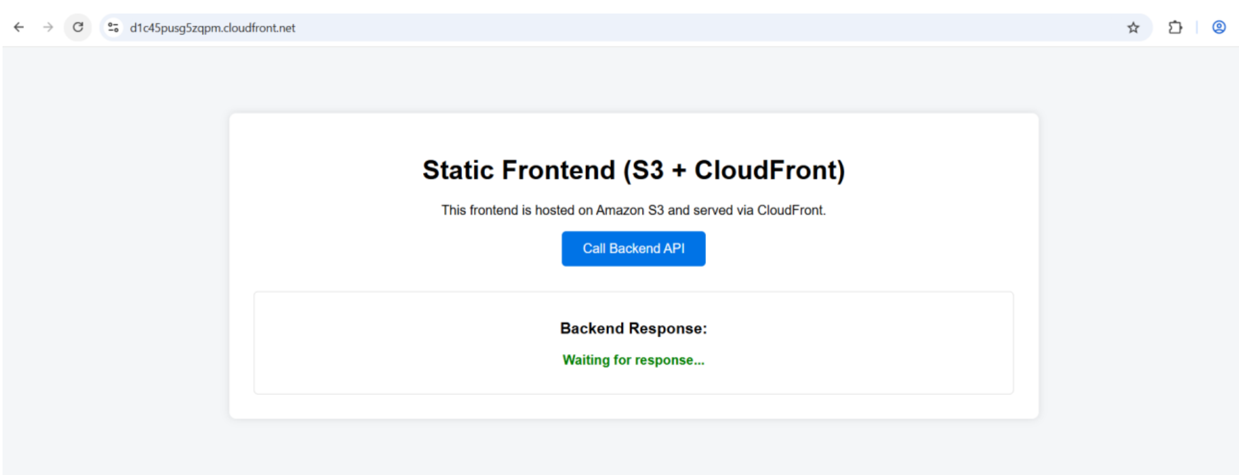
```
[ec2-user@ip-10-0-2-96 ~]$ curl  http://taskappjs-1714280081.us-east-1.elb.amazonaws.com/api
{"message":"Backend is working"}[ec2-user@ip-10-0-2-96 ~]$
```

Also check with only ALB-DNS

curl http://<ALB-DNS>

```
[ec2-user@ip-10-0-2-96 ~]$ curl http://localhost:3000
Backend is running[ec2-usercurl http://taskappjs-1714280081.us-east-1.elb.amazonaws.com.elb.amazonaws.com
Backend is running[ec2-user@ip-10-0-2-96 ~]$
```

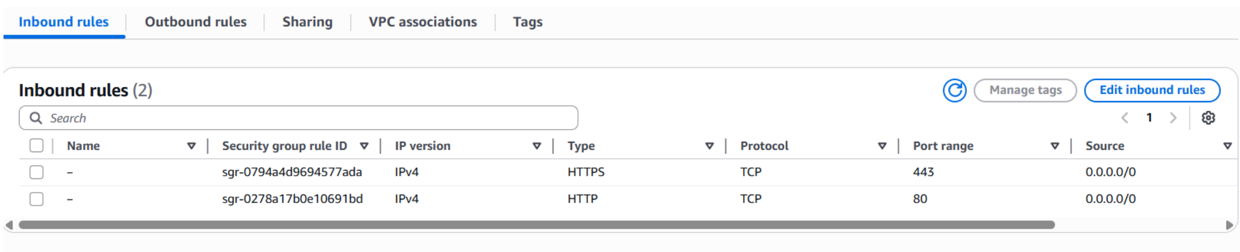
For front end:



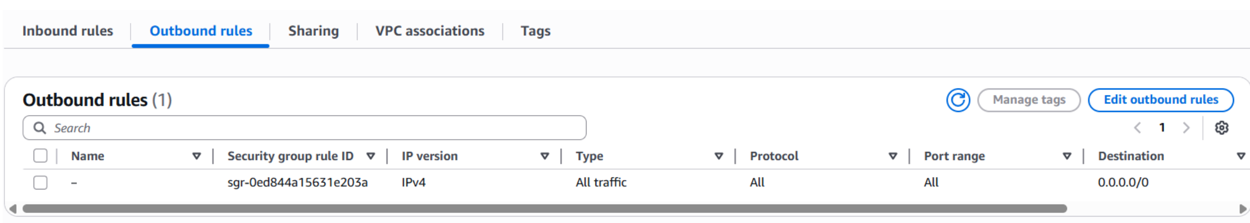
Must check security groups :

ALB Security group

Inbound :

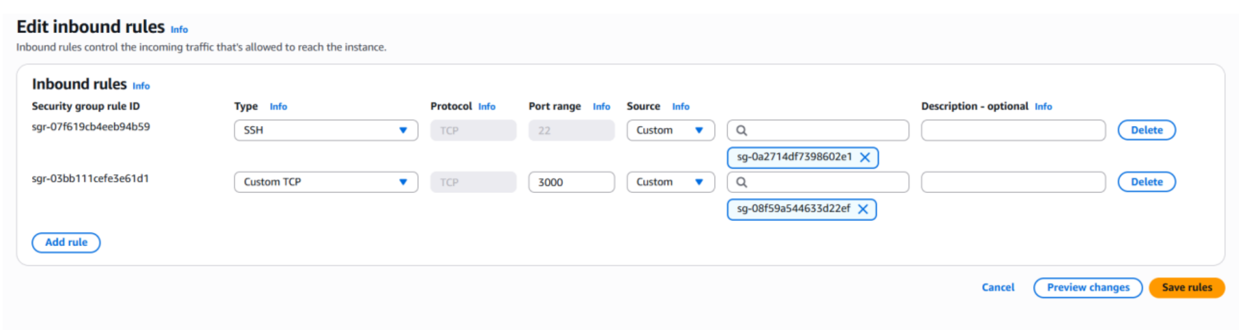


Outbound:



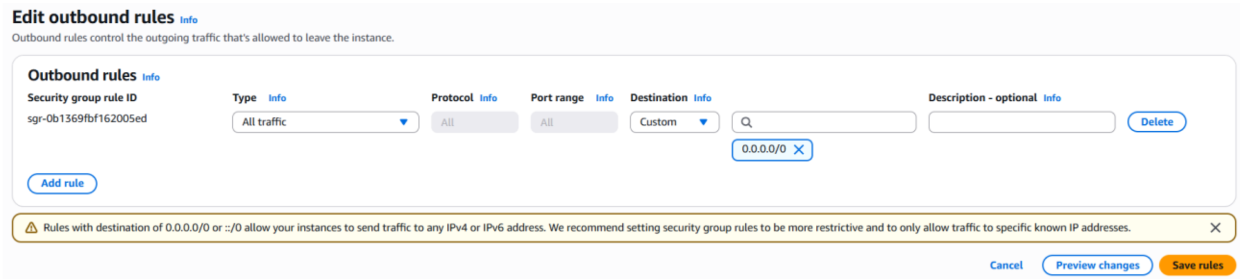
Ec2 Security Group:

Inbound:



Must have to select your ALB security group.

outbound:


Edit outbound rules Info
Outbound rules control the outgoing traffic that's allowed to leave the instance.
Outbound rules Info
Security group rule ID
sgr-0b1369fbf162005ed
Type Info
All traffic
Protocol Info
All
Port range Info
All
Destination Info
Custom
0.0.0.0/0
Description - optional Info
Delete
Add rule
Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.
Cancel
Preview changes
Save rules